

GRUPO I - 1) `static double[] produtoVetor (int[][] m, int[] v)`

O que pedem: devolver um vetor com uma posição por linha de `m`.

Na linha `k`, fazes:

1. multiplicar **cada elemento** da linha `k` por `v[k]`
2. fazer a **média** desses produtos.

Passos

1. `res` tem tamanho `m.length` (nº de linhas)
2. Para cada linha `k`:
 - o somar `m[k][j] * v[k]` para todas as colunas `j`
 - o dividir pelo nº de colunas `m[k].length`
 - o guardar em `res[k]`

```
static double[] produtoVetor(int[][] m, int[] v) {
    double[] res = new double[m.length];

    for (int k = 0; k < m.length; k++) {
        int soma = 0;
        for (int j = 0; j < m[k].length; j++) {
            soma += m[k][j] * v[k];
        }
        res[k] = (double) soma / m[k].length;
    }
    return res;
}
```

2) Classe `GrandPrix`

Pede a classe `GrandPrix` com os métodos e um `toString()` para dar o output mostrado.

O que guardar (atributos)

- `place` (String)
- `drivers` (List<Driver>) — já vem ordenada por ranking, só guardas.

`howManyOfTeam(Team t)`

1. contador = 0
2. percorre drivers
3. se `d.team() == t` contador++

`hasMoreDrivers()`

1. contar quantos drivers há de cada equipa
2. devolver a equipa com maior contagem (o enunciado diz que é única)

`driverTeam(String name)`

1. procurar driver com `d.name().equals(name)`
2. devolver `d.team()`
3. se não encontrar → `null`

```
public class GrandPrix {
    private String place;
    private List<Driver> drivers;

    public GrandPrix(String place, List<Driver> drivers) {
        this.place = place;
        this.drivers = new ArrayList<>(drivers);
    }

    public int howManyOfTeam(Team t) {
        int count = 0;
        for (Driver d : drivers) {
            if (d.team() == t) count++;
        }
        return count;
    }
}
```

```

public Team hasMoreDrivers() {
    Map<Team, Integer> counts = new HashMap<>();
    for (Driver d : drivers) {
        counts.put(d.team(), counts.getOrDefault(d.team(), 0) + 1);
    }

    Team best = null;
    int bestCount = -1;
    for (Map.Entry<Team, Integer> e : counts.entrySet()) {
        if (e.getValue() > bestCount) {
            bestCount = e.getValue();
            best = e.getKey();
        }
    }
    return best;
}

public Team driverTeam(String name) {
    for (Driver d : drivers) {
        if (d.name().equals(name)) return d.team();
    }
    return null;
}

@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append("Place: ").append(place).append("\n");
    sb.append("Drivers: ").append("\n");
    for (Driver d : drivers) {
        sb.append(d.toString()).append("\n"); // Driver já imprime "Name: ... Team: ..."
    }
    return sb.toString();
}
}

```

GRUPO II — GuessingGameLTI

A) **AbstractGame** (classe abstrata)

O que guardar

- `minGuess`, `maxGuess`
- `Random rand`
- `int hiddenValue` (o valor “escondido”)
- lista de jogadores `List<Player> players`
- índice do jogador corrente `int idx`

Métodos pedidos (simples)

- `minGuess()` e `maxGuess()` → devolvem os limites
- `registerPlayer(name)` → `players.add(new Player(name))`
- `numberOfPlayers()` → `players.size()`
- `currentPlayer()` → `players.get(idx).name()`
- `hasGuessed(guess)` → `guess == hiddenValue`
- `betterGuess(g1, g2)`:
 1. `d1 = abs(g1-hidden)`, `d2 = abs(g2-hidden)`
 2. se `d1 < d2` → 1
 3. se `d1 == d2` → 0
 4. senão → -1
- `nextMove()`:
 1. jogador atual = `players.get(idx)`
 2. `guess = jogador.makeGuess(minGuess, maxGuess, rand)`
 3. avançar `idx` (circular)
 4. devolver `guess`

```

public abstract class AbstractGame implements Game {
    private int minGuess, maxGuess;
    private Random rand;
    private int hiddenValue;

    private List<Player> players = new ArrayList<>();
    private int idx = 0;

    public AbstractGame(int minGuess, int maxGuess, Random rand) {
        this.minGuess = minGuess;
        this.maxGuess = maxGuess;
        this.rand = rand;
        this.hiddenValue = rand.nextInt(maxGuess - minGuess + 1) + minGuess;
    }

    @Override public int minGuess() { return minGuess; }
    @Override public int maxGuess() { return maxGuess; }

    @Override
    public void registerPlayer(String name) {
        players.add(new Player(name));
    }

    @Override
    public int numberOfPlayers() {
        return players.size();
    }

    @Override
    public String currentPlayer() {
        return players.get(idx).name();
    }

    @Override
    public boolean hasGuessed(int guess) {
        return guess == hiddenValue;
    }

    @Override
    public int betterGuess(int guess1, int guess2) {
        int d1 = Math.abs(guess1 - hiddenValue);
        int d2 = Math.ensurePositive(Math.abs(guess2 - hiddenValue)); // se não tiveres, usa Math.abs
        if (d1 < d2) return 1;
        if (d1 == d2) return 0;
        return -1;
    }

    @Override
    public int nextMove() {
        Player p = players.get(idx);
        int g = p.makeGuess(minGuess, maxGuess, rand);

        idx = (idx + 1) % players.size();
        return g;
    }
}

```

B) FirstGuessGame

Ideia: joga um palpite de cada vez (um jogador por jogada) e termina quando alguém acerta.

O que guardar

- `boolean finished`
- `String winnerName`

Passos

- `finished()` → devolve `finished`
- `winner()` → devolve `winnerName`
- `nextMove()`:

1. chamar `super.nextMove()` para obter o palpite do jogador corrente
2. se `hasGuessed(palpite)` então:
 - `finished = true`
 - `winnerName = nome do jogador que acabou de jogar`
3. devolver o palpite

```
public class FirstGuessGame extends AbstractGame {
    private boolean done = false;
    private String winner = null;

    public FirstGuessGame(int minGuess, int maxGuess, Random rand) {
        super(minGuess, maxGuess, rand);
    }

    @Override
    public boolean finished() {
        return done;
    }

    @Override
    public String winner() {
        return winner;
    }

    @Override
    public int nextMove() {
        String who = currentPlayer();
        int g = super.nextMove();

        if (hasGuessed(g)) {
            done = true;
            winner = who;
        }
        return g;
    }
}
```

C) OneRoundGame

Ideia: há uma **ronda completa**: todos os jogadores fazem 1 palpite.

O que guardar

- `boolean done`
- `String winnerName`

Passos

- `nextMove()`:
 1. fazer N jogadas (`N = numberOfPlayers()`), chamando `super.nextMove()` N vezes
 2. guardar o melhor palpite e o vencedor (usando `betterGuess`)
 3. no fim marcar `done = true`
 4. devolver o melhor palpite (como pede o enunciado)
- `finished()` → `done`
- `winner()` → `winnerName`

```
public class OneRoundGame extends AbstractGame {
    private boolean done = false;
    private String winner = null;

    public OneRoundGame(int minGuess, int maxGuess, Random rand) {
        super(minGuess, maxGuess, rand);
    }

    @Override
    public boolean finished() {
        return done;
    }
}
```

```

@Override
public String winner() {
    return winner;
}

@Override
public int nextMove() {
    int n = numberOfPlayers();

    int bestGuess = 0;
    String bestPlayer = null;

    for (int i = 0; i < n; i++) {
        String who = currentPlayer();
        int g = super.nextMove();

        if (i == 0) {
            bestGuess = g;
            bestPlayer = who;
        } else {
            int cmp = betterGuess(g, bestGuess);
            if (cmp == 1) { // g é melhor
                bestGuess = g;
                bestPlayer = who;
            }
            // se cmp==0 (empate), fica o primeiro registrado (não troca)
        }
    }

    done = true;
    winner = bestPlayer;
    return bestGuess;
}
}

```

GRUPO III - III.2) **allGamesGuessValues**

O que pedem: para cada jogo da coleção, imprimir **minGuess** e **maxGuess**.

```

static void allGamesGuessValues(Iterable<? extends Game> games) {
    for (Game g : games) {
        System.out.println("minGuess=" + g.minGuess() + " maxGuess=" + g.maxGuess());
    }
}

```